Research Notes on Autoencoder

Autoencoder

>> Section 1

Machine translation Autoencoders have been applied to machine translation, which is usually referred to as neural machine translation (NMT). Unlike traditional autoencoders, the output does not match the input - it is in another language. In NMT, texts are treated as sequences to be encoded into the learning procedure, while on the decoder side sequences in the target language(s) are generated. Language-specific autoencoders incorporate further linguistic features into the learning procedure, such as Chinese decomposition features. Machine translation is rarely still done with autoencoders, due to the availability of more effective transformer networks.

>> Section 2

Definition An autoencoder is defined by the following components: Two sets: the space of encoded messages Z {\displaystyle {\mathcal {Z}}} ; the space of decoded messages X {\displaystyle {\mathcal {X}}} . Typically X {\displaystyle {\mathcal {X}}} and Z {\displaystyle {\mathcal {Z}}} are Euclidean spaces, that is, X = R m , Z = R n {\displaystyle {\mathcal {X}}=\mathbb {R} ^{m},{\mathcal {Z}}=\mathbb {R} ^{n}} with m > n . {\displaystyle m>n.} Two parametrized families of functions: the encoder family E φ : X → Z {\displaystyle E_{\phi }:{\mathcal {X}}\rightarrow {\mathcal {Z}}} , parametrized by φ {\displaystyle \phi } ; the decoder family D θ : Z → X {\displaystyle D_{\theta }:{\mathcal {Z}}\rightarrow {\mathcal {X}}} , parametrized by θ {\displaystyle \theta } .For any x ∊ X {\displaystyle x\in {\mathcal {X}}} , we usually write z = E φ (x) {\displaystyle z=E_{\phi }(x)} , and refer to it as the code, the latent variable, latent representation, latent vector, etc. Conversely, for any z ∊ Z {\displaystyle z\in {\mathcal {Z}}} , we usually write x ′ = D θ (z) {\displaystyle x'=D_{\theta }(z)} , and refer to it as the (decoded) message. Usually, both the encoder and the decoder are defined as multilayer perceptrons (MLPs). For example, a one-layer-MLP encoder E φ {\displaystyle E_{\phi }} is:

>> Section 3

Depth can exponentially reduce the computational cost of representing some functions. Depth can exponentially decrease the amount of training data needed to learn some functions. Experimentally, deep autoencoders yield better compression compared to shallow or linear autoencoders. Depth allows for advantages over traditional methods as one can show that after training single layer linear autoencoders have a latent space whose vectors span the same subspace as the eigenvectors found in Principal component analysis.

>> Section 4

additive isotropic Gaussian noise, masking noise (a fraction of the input is randomly chosen and set to 0) salt-and-pepper noise (a fraction of the input is randomly chosen and randomly set to its minimum or maximum value).

>> Section 5

Contractive autoencoder (CAE) A contractive autoencoder (CAE) adds the contractive regularization loss to the standard autoencoder loss: min θ , φ L (θ , φ) + λ L cont (θ , φ) {\displaystyle \min _{\theta ,\phi }L(\theta ,\phi)+\lambda L_{\text{cont}}(\theta ,\phi)} where λ > 0 {\displaystyle \lambda >0} measures how much contractive-ness we want to enforce. The contractive regularization loss itself is defined as the expected square of Frobenius norm of the Jacobian matrix of the encoder activations with respect to the input: L cont (θ , φ) = E x ∼ μ r e f ■ ∇ x E φ (x) ■ F 2 {\displaystyle L_{\text{cont}}(\theta ,\phi)=\mathbb {E} _{x\sim \mu _{ref}}\|\nabla _{x}E_{\phi }(x)\|_{F}^{2}} To understand what L cont {\displaystyle L_{\text{cont}}} measures, note the fact ■ E φ (x + δ x) − E φ (x) ■ 2 ≤ ■ ∇ x E φ (x) ■ F ■ δ x ■ 2 {\displaystyle \|E_{\phi }(x+\delta x)-E_{\phi }(x)\|_{2}\leq \|\nabla _{x}E_{\phi }(x)\|_{F}\|\delta x\|_{2}} for any message x ∊ X {\displaystyle x\in {\mathcal {X}}} , and small variation δ x {\displaystyle \delta x} in it. Thus, if ■ ∇ x E φ (x) ■ F 2 {\displaystyle \|\nabla _{x}E_{\phi }(x)\|_{F}^{2}} is small, it means that a small neighborhood of the message maps to a small neighborhood of its code. This is a desired property, as it means small variation in the message leads to small, perhaps even zero, variation in its code, like how two pictures may look the same even if they are not exactly the same. The DAE can be understood as an infinitesimal limit of CAE: in the limit of small Gaussian input noise, DAEs make the reconstruction function resist small but finite-sized input perturbations, while CAEs make the extracted features resist infinitesimal input perturbations.

>> Section 6

Dimensionality reduction Dimensionality reduction was one of the first deep learning applications. For Hinton's 2006 study, he pretrained a multi-layer autoencoder with a stack of RBMs and then used their weights to initialize a deep autoencoder with gradually smaller hidden layers until hitting a bottleneck of 30 neurons. The resulting 30 dimensions of the code yielded a smaller reconstruction error compared to the first 30 components of a principal component analysis (PCA), and learned a representation that was qualitatively easier to interpret, clearly separating data clusters. Reducing dimensions can improve performance on tasks such as classification. Indeed, the hallmark of dimensionality reduction is to place semantically related examples near each other.

>> Section 7

Image processing The characteristics of autoencoders are useful in image processing. One example can be found in lossy image compression, where autoencoders outperformed other approaches and proved competitive against JPEG 2000. Another useful application of autoencoders in image preprocessing is image denoising. Autoencoders found use in more demanding contexts such as medical imaging where they have been used for image denoising as well as super-resolution. In image-assisted diagnosis, experiments have applied autoencoders for breast cancer detection and for modelling the relation between the cognitive decline of Alzheimer's disease and the latent features of an autoencoder trained with MRI.

>> Section 8

s (ρ , ρ ^) = K L (ρ | | ρ ^) = ρ log ■ ρ ρ ^ + (1 − ρ) log ■ 1 − ρ 1 − ρ ^ {\displaystyle s(\rho ,{\hat {\rho }})=KL(\rho ||{\hat {\rho }})=\rho \log {\frac {\rho }{\hat {\rho }}}+(1-\rho)\log {\frac {1-\rho }{1-{\hat {\rho }}}}}